ECE 385
Spring 2025

Experiment #5

An 8-Bit Multiplier in System Verilog
Lab-5

Cristina Natalia Lumban Tobing 3220300398
Artha Amanda Sidauruk 3220300399
TA: Qiyang Wu

Table of Contents

1. Introduction 3

1.1 Summarize the basic functionality of the multiplier circuit 3

2. Pre-Lab Question 3

2.1 Rework the multiplication example on page 5.2 of the lab manual, as in computing 00000111 * 11000101 in a table similar to the example 3

3. Written description and diagrams of multiplier circuit 4

3.1 Summary of operation 4

3.2 Top Level Block Diagram 5

3.3 Written Description of .sv Modules 5

3.4 State Diagram for Control Unit 9

4. Annotated pre-lab simulation waveforms 9

4.1 Create your own testbench for your multiplier which displays the following 9

4.1.1 Four operations where operands have signs (+*+), (+*-), (-*+), and (-*-) 9

4.1.2 Waveform must have notes that clearly show the operands as well as the result, etc. 9

5. Answers to two post-lab questions 11

5.1 Fill in the table shown in 5.6 with your signal design statistics 11

5.2 Write down several ideas on how the maximum frequency of your design could be increased or the gate count could be decreased 11

5.3 Make sure the lab report answers at least the following questions: 11

- What is the purpose of the X register. When does the X register get set/cleared? 11
- What are the limitations of continuous multiplications? Under what circumstances will the implemented algorithm fail? 11
- What are the advantages (and disadvantages?) of the implemented multiplication algorithm over the pencil-and-paper method discussed in the introduction? 12

6. Conclusion 12

6.1 Discuss functionality of your design. If parts of your design didn't work, discuss what could be done to fix it. 12

6.2 Was there anything ambiguous, incorrect, or unnecessarily difficult in the lab manual or given materials which can be improved for next semester? You can also specify what we did right so it doesn't changed. 12

1. Introduction

1.1 Summarize the basic functionality of the multiplier circuit

We are designing multiplier in System Verilog for two 8-bit 2's compliment numbers and then run that multiplier into the DE2 FPGA board for this week's lab experiment. To make this multiplier, we are using add-shift method with a Moore machine with 17 states and a 9-bit adder/subtractor.

2. Pre-Lab Question

2.1 Rework the multiplication example on page 5.2 of the lab manual, as in computing 00000111 * 11000101 in a table similar to the example

Here we are implementing B = 0000 0111, (7 in binary) as the multiplier and S = 1100 0101, (-59 in binary) as the multiplicand.

Function	X	A	B	M	Comments for the next step
Clear A, Load B	0	0000 0000	0000 0111	1	Add S to A, since M = 1
ADD	1	1100 0101	0000 0111	1	Shift XAB by one bit after ADD complete
SHIFT	1	1110 0010	1000 0011	1	Add S to A, since M = 1
ADD	1	1010 0111	1000 0011	1	Shift XAB by one bit after ADD complete
SHIFT	1	1101 0011	1100 0001	1	Add S to A, since M = 1
ADD	1	1001 1000	1100 0001	1	Shift XAB by one bit after ADD complete
SHIFT	1	1100 1100	0110 0000	0	Do not add S to A, since M = 0. Shift XAB
SHIFT	1	1110 0110	0011 0000	0	Do not add S to A, since M = 0. Shift XAB
SHIFT	1	1111 0011	0001 1000	0	Do not add S to A, since M = 0. Shift XAB
SHIFT	1	1111 1001	1000 1100	0	Do not add S to A, since M = 0. Shift XAB
SHIFT	1	1111 1100	1100 0110	0	Do not add S to A, since M = 0. Shift XAB
SHIFT	1	1111 1110	0110 0011	1	8th shift done. Stop. 16-bit Product in AB

Table 1: Computing Multiplication

3. Written description and diagrams of multiplier circuit

3.1 Summary of operation

- **Clear and Load**
  - Purpose: Initialize registers and load the multiplier.
  - Process:
    - The multiplier is set using switches (S).
    - When the Clear_A_Load_B button is pressed:
    - The multiplier is loaded into Register B.
    - The X and A registers are cleared to ensure a fresh computation.
    - Next, the multiplicand is set using switches.
    - Pressing the Run button starts the multiplication process.
- **ADD Computation and SHIFT**

  ADD State

  The multiplication process checks the least significant bit (LSB) of Register B (M):
  - If M = 1 (LSB is 1), then:
    - The multiplicand (S) (extended to 9 bits) is added or subtracted to/from Register A, depending on the sign bit (X).
    - The least 8 bits of the result are stored back in Register A.
    - The first bit of the result is stored in Register X (to handle sign extension in signed multiplication).
  - If M = 0, no addition is performed.

  SHIFT State

  After the ADD state, the system enters the SHIFT state:
  - The entire XAB (17-bit value) is arithmetically right-shifted by one bit.
  - This is equivalent to dividing the value by 2, which is needed for the multiplication process.
  - The system alternates between ADD and SHIFT states until 16 SHIFT operations have been performed (one per bit in the multiplier).
  - After 16 shifts, the multiplication is complete, and the result is stored in AB.
- **Store**

  Once 16 shifts have been completed:
  - The multiplication is finished.
  - The product (result) is stored in AB.
  - The result is displayed on a Hex display for visualization.

3.2 Top Level Block Diagram

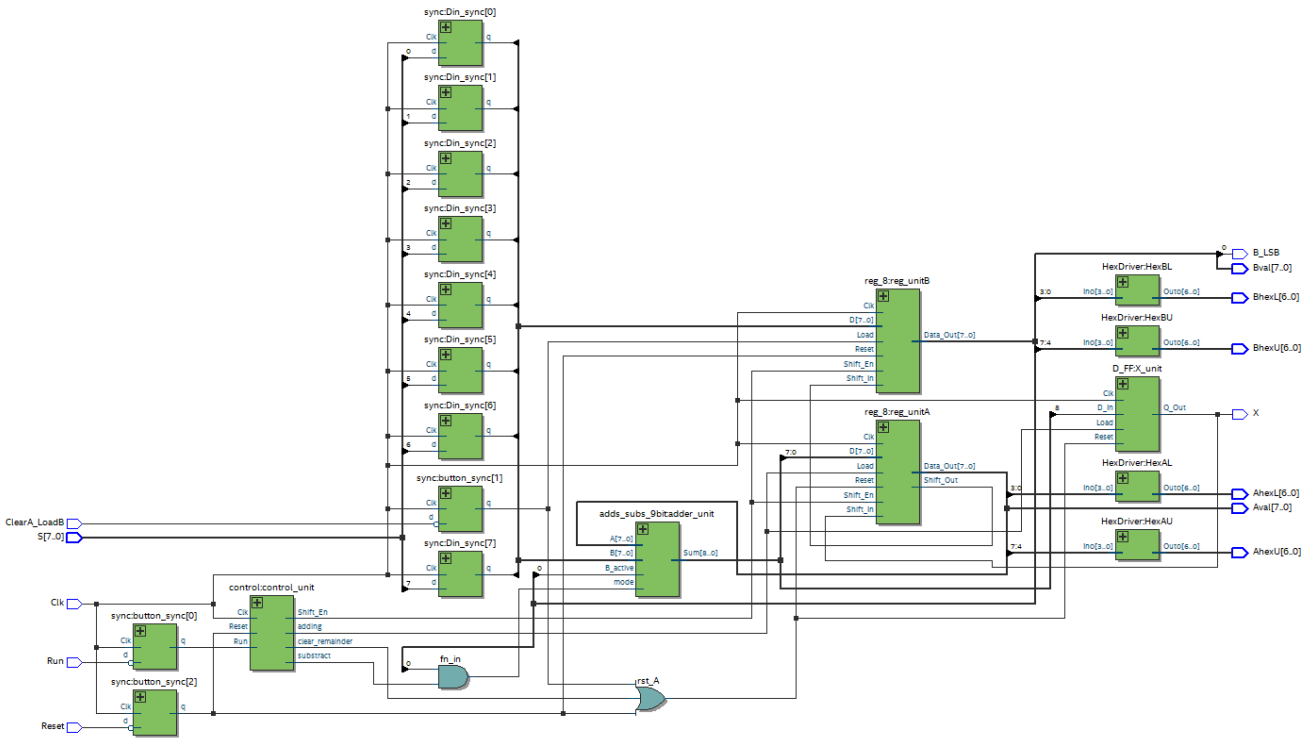

Figure 1: Top Level Block Diagram

3.3 Written Description of .sv Modules

Module: adds_subs_9bit.sv

Inputs: [7:0] A, [7:0] B, fn, M

Outputs: [8:0] Sum, CO

Description: The module contains a 4-bit full adder as a submodule. It can perform addition (A + B), subtraction (A - B), and no operation. The selection of operations is controlled by input signals, which are fn (to determine whether the operation is addition or subtraction) and M (to determines whether to perform an operation between A and B or not.)

Purpose: To implement an arithmetic unit that can add or subtract two 4-bit numbers (A and B). The operation is controlled by the input signals (fn and M).

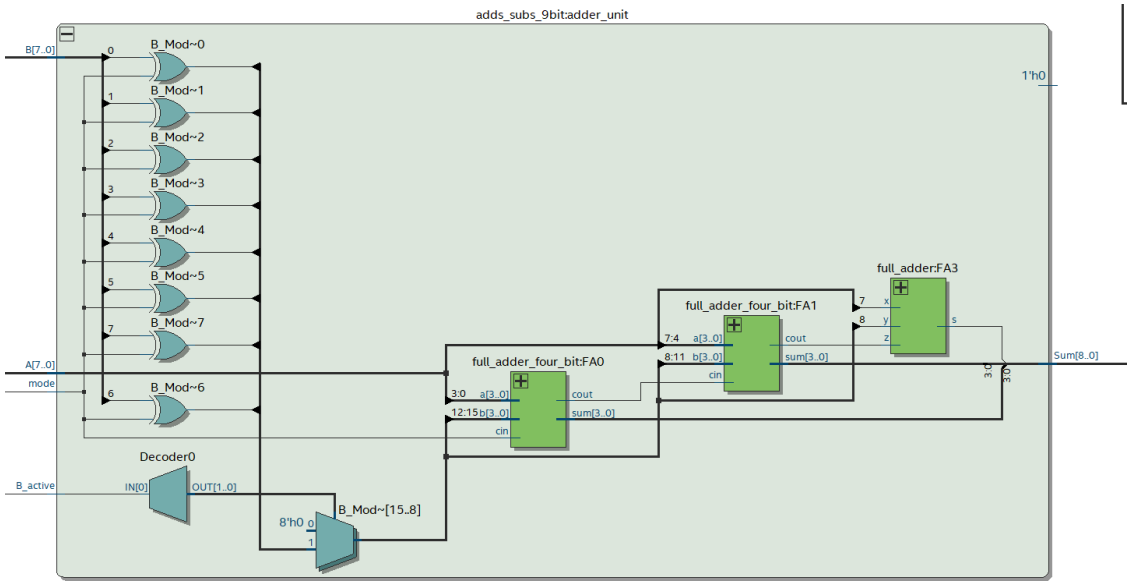

Figure 2: Adds_subs_9bit Block Diagram

Module:	sync.sv
Inputs:	clk, Reset, d
Outputs:	q
Description:	An ‘always_ff’ block is used to bring asynchronous signals into the FPGA
Purpose:	These are synchronizers required for synchronizing the input siganl from button or switches.

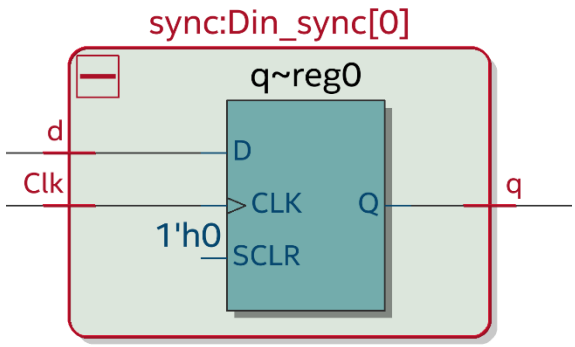

Figure 3: sync.sv Block Diagram

Module: control.sv
Inputs: Clk, Reset, Run
Outputs: Shift_En, adding, subtract, clear_reminder
Description: This module use a Finite State Machine to build a control unit.
Purpose: Accepting input signal from user and give instructions to other unit in the circuit

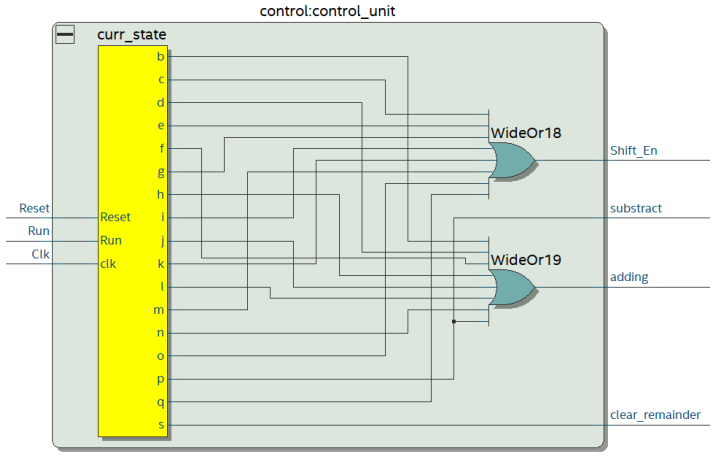

Figure 4: control.sv Block Diagram

Module: HexDriver.sv
Inputs: [3:0]In0
Outputs: [6:0]Out0
Description: This translate the data to the form that a LED screen can recognize
Purpose: To display the value of register A and B onto the LED screen of the FPGA board.

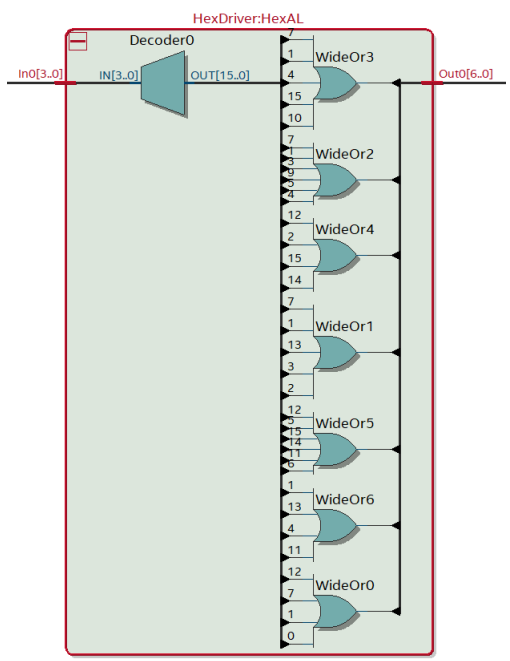

Figure 5: Hex Driver Block Diagram

Module: D_FF.sv
Inputs: clk, Load, Reset, D_In
Outputs: Q_Out
Description: This is a D-flip-flop which can store one bit.
Purpose: To store the sign bit of A.

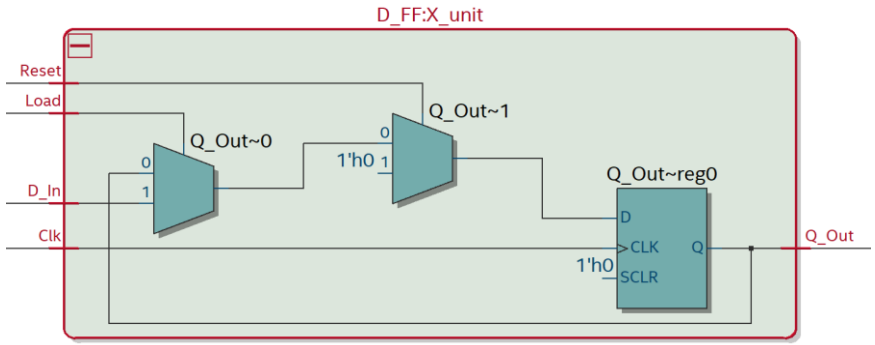

Figure 6: D_FF Block Diagram

Module: reg_8.sv
Inputs: Clk, Reset, Load, D[7:0], Shift_En, Shift_In
Outputs: Shift_Out, Data_Out[7:0]
Description: This is a positive-edge triggered 8-bit register with asynchronous reset and synchronous load.
Purpose: To create the registers that store operands A and B in the adder circuit

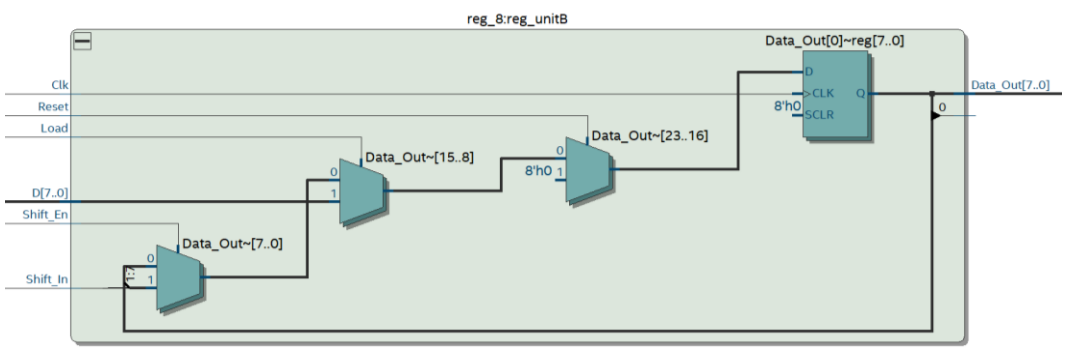

Figure 7: reg_8 Block Diagram

3.4 State Diagram for Control Unit

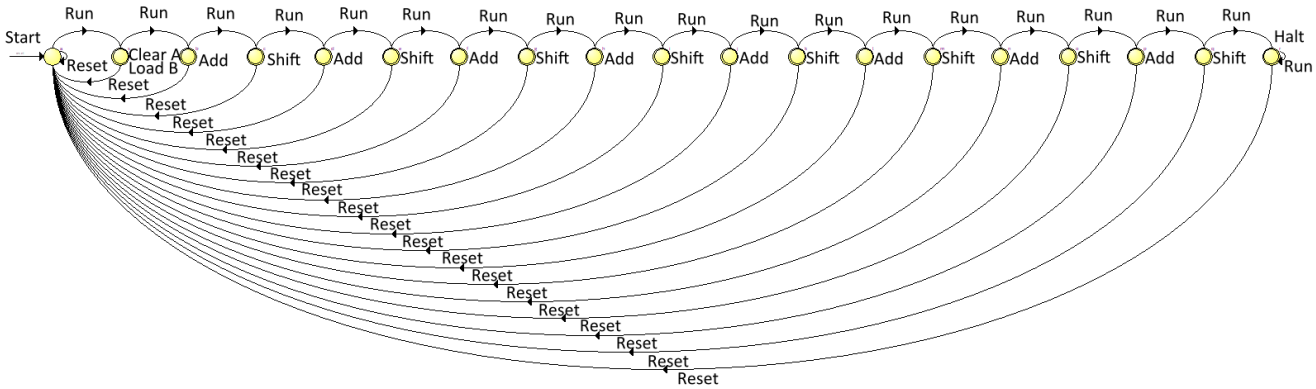

Figure 8: State Diagram for Control Unit

4. Annotated pre-lab simulation waveforms

4.1 Create your own testbench for your multiplier which displays the following

4.1.1 Four operations where operands have signs (+*+), (+*-), (-*+), and (-*-)

4.1.2 Waveform must have notes that clearly show the operands as well as the result, etc.

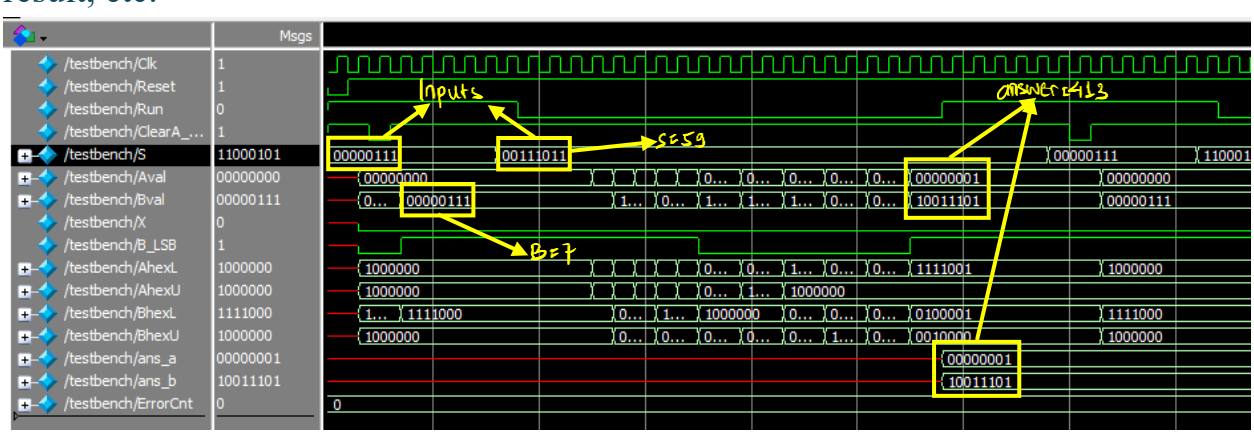

Figure 9: Simulation for ++*

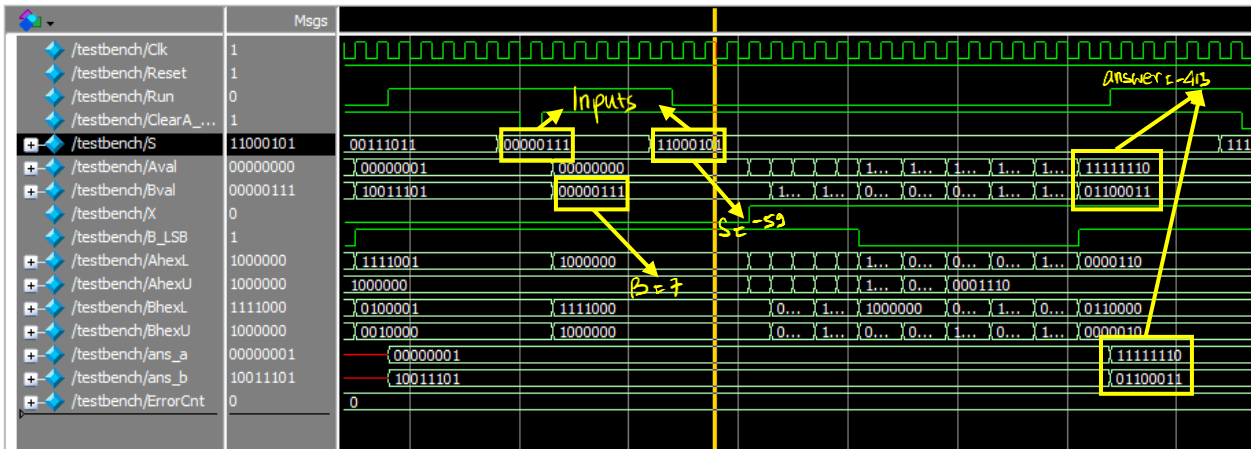

Figure 10: Simulation for -+*

.

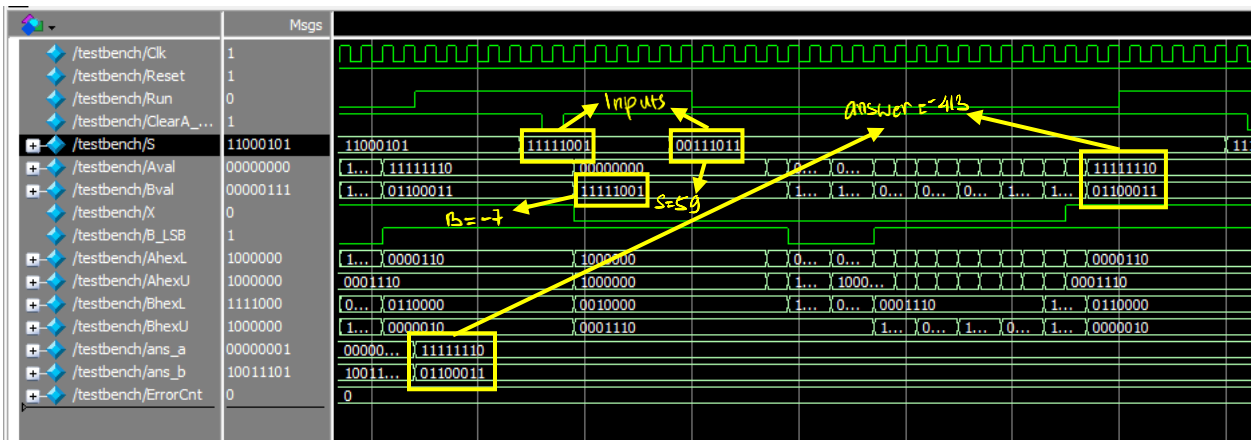

Figure 11: Simulation for +-*

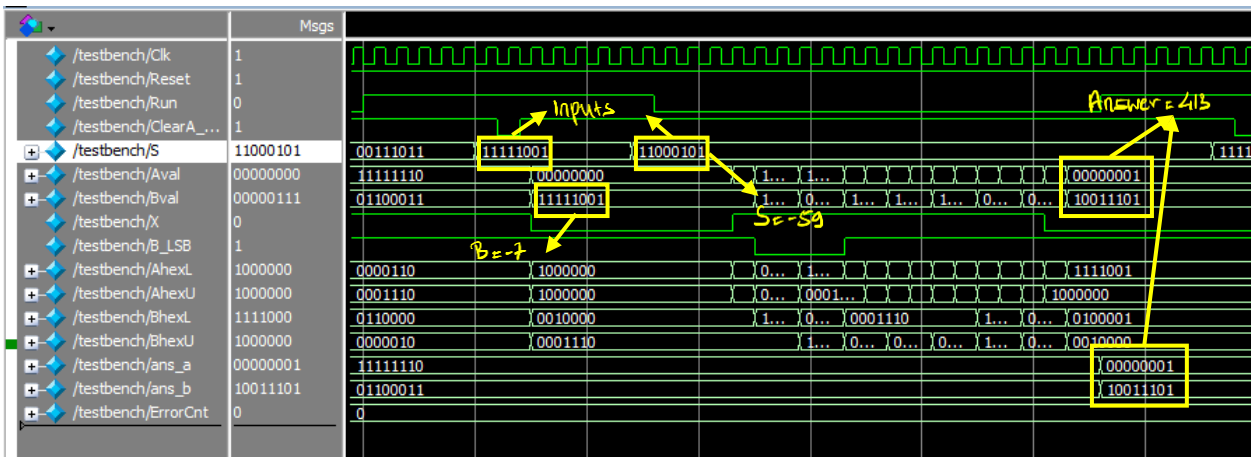

Figure 12: Simulation for --*

5. Answers to two post-lab questions

5.1 Fill in the table shown in 5.6 with your signal design statistics

LUT	98
DSP	0
Memory (BRAM)	0
Flip-Flop	47
Frequency	72.05 MHz
Static Power	98.52 mW
Dynamic Power	3.13 mW
Total Power	152.28 mW

Table 2: Design Statistic Table

5.2 Write down several ideas on how the maximum frequency of your design could be increased or the gate count could be decreased

- Increasing the Maximum Frequency: To achieve a higher maximum frequency, we can substitute our carry-ripple adder with a more efficient carry-select or carry-lookahead adder. These alternatives help minimize delays by enabling partial computations to be executed simultaneously.
- Decreasing the Gate Count: To reduce the total gate count, we can merge the 8-bit register (A) with a single flip-flop (X) to form a 9-bit register. This integration eliminates the need for extra gates dedicated to the flip-flop, thereby reducing overall gate usage.

5.3 Make sure the lab report answers at least the following questions:

- ### What is the purpose of the X register. When does the X register get set/cleared?

The X register preserves the correct sign during shift operations by extending the sign bit of result. It updates when the most significant bit of A changes after an addition or subtraction. Additionally, it is cleared when the Clear_A_load_B function is activated.

- ### What are the limitations of continuous multiplications? Under what circumstances will the implemented algorithm fail?

Continuous multiplications are limited by overflow, as the result can require more bits than the system can store, leading to data loss. The algorithm fails when the result exceeds 8 bits because the A register is cleared before each new multiplication, resetting any overflowed bits to zero. Additionally, the add-shift method becomes inefficient for larger numbers, as it requires multiple iterations, slowing down computation.

- What are the advantages (and disadvantages?) of the implemented multiplication algorithm over the pencil-and-paper method discussed in the introduction?

Advantages: The implemented multiplication algorithm is automated, making it much faster and more accurate than the manual pencil-and-paper method. It efficiently uses hardware resources, requiring only two registers and one flip-flop to store intermediate values, which saves space. Additionally, it eliminates the need for repeated manual additions, significantly improving computational speed.

Disadvantages: The algorithm is less intuitive than the pencil-and-paper method and does not allow the operands to be easily viewed once the calculation is complete. It also has hardware complexity, requiring more gates and registers, and increasing cost. Furthermore, it has limitations in continuous multiplications, as results exceeding 8 bits are lost.

6. Conclusion

6.1 Discuss functionality of your design. If parts of your design didn't work, discuss what could be done to fix it.

The design on this lab is for multiplying two 8-bit 2's compliment numbers and then run it on to the DE2 FPGA board. We met a difficulty when installing the USB Blaster Driver from the DE2 FPGA board on Cristina's laptop. We tried to solve this problem by following several tutorials from the internet but still couldn't find the solution to this problem. We send the code to Artha's laptop which doesn't have this problem and use it for demo. We are still trying to fix the problem with Cristina's laptop.

6.2 Was there anything ambiguous, incorrect, or unnecessarily difficult in the lab manual or given materials which can be improved for next semester? You can also specify what we did right so it doesn't changed.

The lab manual is straight forward and easy to follow. The difficulties are also reasonable for undergraduate level like us; however, the lab manual would be better if there are a more specific explanation of what to do on the lab because from our experience, we used a pretty long time to figure out what to do on this lab. The open lab session is really helping on doing this lab.